

Placement Portal Application

Project Report

Submitted By

Kunal Sachin Kandale

Roll No: 24F3002856

Email: 24f3002856@ds.study.iitm.ac.in

Course: Modern Application Development 2



Author

I am a B.Tech student specializing in Electronics and Communication Engineering with a strong interest in full-stack web development, backend systems, and scalable application design. This project demonstrates my understanding of modern web application development using a RESTful architecture with Flask for the backend and Vue.js for the frontend. It also incorporates asynchronous task processing, caching, and role-based access control to build a complete placement management system.

AI/LLM Usage

AI tools such as ChatGPT were used during the development of this project primarily for conceptual understanding, debugging, and improving code structure.

The approximate usage of AI assistance is around 20–25%, mainly limited to guidance and optimization rather than direct code generation.

Project Description

- **Project title:** Placement Portal Application
- **Problem Statement:**
To develop a web-based Placement Portal that streamlines the campus recruitment process by providing a centralized platform for students, companies, and administrators. The system simplifies placement drive management, application tracking, interview scheduling, offer management, and administrative approvals.
- **Approach:**
The application was built using Flask as the backend framework and Vue.js as the frontend following a RESTful API architecture. It enables role-based access for students, companies, and administrators, while Redis improves performance through caching and Celery handles background tasks such as email reminders, monthly reports, CSV export, and automatic updates for drives and interviews.

Technologies Used

- **Flask** – Backend framework for developing RESTful APIs and application logic.
- **Vue.js** – Frontend framework for building a responsive Single Page Application (SPA).
- **SQLite** – Relational database for storing application data.
- **Flask-SQLAlchemy** – ORM for database modeling and management.
- **Flask-Security** – Authentication and role-based access control.

- **Flask-RESTful** – Development of REST APIs for frontend-backend communication.
- **Redis** – API caching to improve application performance.
- **Celery** – Asynchronous task processing and scheduled background jobs.
- **Bootstrap** – Responsive UI design and styling.

These technologies were selected to build a scalable, efficient, and user-friendly placement management system.

Database Schema Design

The database is designed using normalization principles and consists of the following key entities:

- **User & Role:** Handles authentication and role-based access control.
- **Student:** Stores student profile, academic details, skills, and resume.
- **Company:** Maintains company information, approval status, and blacklist status.
- **Placement Drive:** Contains job postings, eligibility criteria, deadlines, and drive status.
- **Application:** Records student applications and recruitment progress.
- **Interview:** Manages interview schedules, mode, meeting links, and status.
- **Placement:** Stores placement details, joining information, and offer status.
- **Offer Letter:** Maintains uploaded offer letter PDFs for selected students.

Key Design Decisions:

- Primary and foreign keys ensure referential integrity between entities.
- Many-to-many relationship is implemented between Users and Roles for authentication.
- Approval, blacklist, and status fields help manage workflows without deleting records.
- The schema is modular and normalized, making it scalable, maintainable, and suitable for RESTful API operations.

System Architecture

The application follows a modular client-server architecture:

- **Flask Backend:** Implements RESTful APIs, authentication, business logic, database operations, and role-based access control.
- **Vue.js Frontend:** Provides a Single Page Application (SPA) with reusable components and client-side routing.
- **SQLite Database:** Stores users, companies, students, drives, applications, interviews, placements, and offer letters.

- **Redis Cache:** Caches frequently accessed API responses to improve performance.
- **Celery Workers:** Execute asynchronous and scheduled tasks such as email reminders, monthly reports, CSV export, and automatic updates.

The database is created programmatically using SQLAlchemy, while the frontend and backend communicate through REST APIs, resulting in a modular, scalable, and maintainable application architecture.

Important API Resource Endpoints

Endpoint	Method	Description
/api/login	POST	Authenticate user and create login session
/api/company/{company_id}	GET	Retrieve company profile
/api/companies	GET	Retrieve list of companies with required filters
/api/company/dashboard	GET	Retrieve company dashboard statistics
/api/student/{student_id}	PUT	Update student profile and upload resume
/api/students	GET	Retrieve list of students with required filters
/api/student/export	POST	Export student applications as CSV
/api/drive	POST	Create a new placement drive
/api/drives	GET	Retrieve list of drives with required filters
/api/application/{id}	PUT	Update application status
/api/applications	GET	Retrieve list of applications with required filters
/api/interview	POST	Schedule an interview
/api/placement/{placement_id}	PUT	Update offer acceptance status
/api/placements	GET	Retrieve list of placements with required filters
/api/offer_letter/{placement_id}	GET	Retrieve uploaded offer letter
/api/offer_letter	POST	Upload offer letter PDF
/api/admin/search	GET	Search companies, students, drives, applications, and placements
/api/admin/student/{student_id}/blacklist	PUT	Blacklist students in case of malpractices.
/api/admin/company/company_id/approve	PUT	Approve Company Registrations by admin

Table 1: Major REST API Endpoints

YAML API Definition File: Included separately in the submission ZIP as `api.yaml`.

Core Features Implemented

Authentication:

- Secure login and registration using hashed passwords for students and companies.
- Role-based access control for Admin, Company, and Student using Flask-Security.
- Session-based authentication with protected REST API endpoints.

Admin Functionalities:

- Dashboard displaying placement statistics.
- Approve or reject company registrations and placement drives.
- Blacklist or whitelist companies and students.
- Search companies, students, drives, applications, and placements.

Company Functionalities:

- Company registration and profile management.
- Create and manage placement drives.
- Review student applications and update application status.
- Schedule interviews and manage placements.
- Upload offer letters for selected students.

Student Functionalities:

- Student registration and profile management with resume upload.
- View and apply for approved placement drives.
- Track application status and interview schedules.
- View, accept, or reject placement offers.
- Export application history as a CSV file.

Additional Functionalities:

- Redis-based API caching for improved performance.
- Celery-powered asynchronous tasks for CSV export.
- Scheduled jobs for daily reminders, monthly reports, automatic drive closure, and interview completion.
- Prevention of duplicate applications and automated status management.

Demo Video

Video Link: [Click here to view the demo video](#)